

VishwaTech-Labs Python String Methods

□ COLOURFUL HANDWRITTEN-STYLE CLASSROOM NOTES □

Learn each method as: INPUT → METHOD → VARIABLE → OUTPUT → REAL-TIME USE.

1. STRING BASICS

A string is a sequence of characters enclosed in quotes. Python's string data type is `str`. Strings are immutable: methods normally return a new value.

```
name = "Vishwanath"
city = "Bengaluru"
result = type(name)
print(result) # <class str>
```

Indexing and slicing:

```
language = "Python"
first = language[0]
last = language[-1]
part = language[0:3]
print(first) # P
print(last) # n
print(part) # Pyt
```

2. STRING METHODS — CASE & CLEANING

1. upper()

Definition: Converts alphabetic characters to uppercase.

Example — assign the result to a variable:

```
name = "vishwa"
result = name.upper()
print(result) # VISHWA
```

Real-time use: Normalize names, labels and user input.

2. lower()

Definition: Converts alphabetic characters to lowercase.

Example — assign the result to a variable:

```
email = "VISHWA@GMAIL.COM"
result = email.lower()
print(result) # vishwa@gmail.com
```

Real-time use: Case-insensitive email/user comparisons.

3. capitalize()

Definition: Makes the first character uppercase and the rest lowercase.

Example — assign the result to a variable:

```
text = "WELCOME TO PYTHON"
result = text.capitalize()
print(result) # Welcome to python
```

Real-time use: Formatting a sentence.

4. title()

Definition: Makes the first character of each word uppercase.

Example — assign the result to a variable:

```
name = "vishwanath gowda"
result = name.title()
print(result) # Vishwanath Gowda
```

Real-time use: Formatting names.

5. swapcase()

Definition: Changes uppercase letters to lowercase and lowercase letters to uppercase.

Example — assign the result to a variable:

```
text = "Hello Python"
result = text.swapcase()
print(result) # hELLO pYTHON
```

Real-time use: Case transformation.

6. strip()

Definition: Removes whitespace or specified characters from both ends.

Example — assign the result to a variable:

```
username = " admin "
clean_username = username.strip()
print(clean_username) # admin
```

Real-time use: Cleaning form/user input.

7. lstrip()

Definition: Removes whitespace or specified characters from the left side.

Example — assign the result to a variable:

```
data = " Python"
result = data.lstrip()
print(result) # Python
```

Real-time use: Cleaning left padding.

8.rstrip()

Definition: Removes whitespace or specified characters from the right side.

Example — assign the result to a variable:

```
data = "Python "
```

Real-time use: Cleaning trailing spaces.

9. replace()

Definition: Returns a new string with matching text replaced.

Example — assign the result to a variable:

```
phone = "987-654-3210"
clean_phone = phone.replace("-", "")
print(clean_phone) # 9876543210
```

Real-time use: Data cleaning and standardization.

10. find()

Definition: Returns the lowest index of a substring; returns -1 if absent.

Example — assign the result to a variable:

```
email = "vishwa@gmail.com"
position = email.find("@")
print(position) # 6
```

Real-time use: Searching emails, logs and text.

11. rfind()

Definition: Returns the highest index of a substring.

Example — assign the result to a variable:

```
path = "/app/logs/app.log"
position = path.rfind("/")
print(position) # 9
```

Real-time use: Finding the last path separator.

12. index()

Definition: Returns the index of a substring; raises ValueError if absent.

Example — assign the result to a variable:

```
text = "Python"
position = text.index("t")
print(position) # 2
```

Real-time use: Use when missing text should be an error.

13. rindex()

Definition: Returns the highest index; raises ValueError if absent.

Example — assign the result to a variable:

```
filename = "backup.2026.csv"
position = filename.rindex(".")
print(position) # 11
```

Real-time use: Finding the final separator.

14. count()

Definition: Counts non-overlapping occurrences of a substring.

Example — assign the result to a variable:

```
log = "ERROR ERROR WARNING ERROR"
error_count = log.count("ERROR")
print(error_count) # 3
```

Real-time use: Counting errors/events in logs.

15. startswith()

Definition: Checks whether a string begins with a prefix.

Example — assign the result to a variable:

```
url = "https://example.com"
is_secure = url.startswith("https")
print(is_secure) # True
```

Real-time use: URL and log classification.

16. endswith()

Definition: Checks whether a string ends with a suffix.

Example — assign the result to a variable:

```
filename = "employee_data.csv"
is_csv = filename.endswith(".csv")
print(is_csv) # True
```

Real-time use: File-type validation.

3. STRING METHODS — SEARCH, SPLIT, JOIN & PARSING

17. split()

Definition: Splits a string into a list using a separator.

Example — assign the result to a variable:

```
data = "Python, AWS, Docker"
parts = data.split(",")
print(parts)
```

Real-time use: Parsing CSV-like data, commands and emails.

18. rsplit()

Definition: Splits from the right; useful when only the final separator matters.

Example — assign the result to a variable:

```
path = "team/project/reports/final.pdf"
parts = path.rsplit("/", 1)
print(parts)
```

Real-time use: Extracting a filename from a path.

19. splitlines()

Definition: Splits a multi-line string into a list of lines.

Example — assign the result to a variable:

```
logs = "INFO: Start\nERROR: Failed\nINFO: End"
lines = logs.splitlines()
print(lines)
```

Real-time use: Processing logs/configuration text.

20. join()

Definition: Combines an iterable of strings using a separator.

Example — assign the result to a variable:

```
skills = ["Python", "AWS", "Docker"]
result = ", ".join(skills)
print(result) # Python, AWS, Docker
```

Real-time use: Building reports and CSV-style output.

21. partition()

Definition: Splits at the first separator and returns before, separator, after.

Example — assign the result to a variable:

```
email = "vishwa@gmail.com"
result = email.partition("@")
print(result)
```

Real-time use: Parsing key/value or email-style data.

22. rpartition()

Definition: Splits at the last separator and returns three parts.

Example — assign the result to a variable:

```
version = "app.release.2026"
result = version.rpartition(".")
print(result)
```

Real-time use: Extracting final version/extension.

23. removeprefix()

Definition: Removes an exact prefix if it exists.

Example — assign the result to a variable:

```
url = "https://example.com"  
result = url.removeprefix("https://")  
print(result) # example.com
```

Real-time use: Normalizing protocol prefixes.

24. removesuffix()

Definition: Removes an exact suffix if it exists.

Example — assign the result to a variable:

```
filename = "report.csv"  
result = filename.removesuffix(".csv")  
print(result) # report
```

Real-time use: Separating filename and extension.

4. STRING METHODS — FORMATTING & ENCODING

25. center()

Definition: Centers a string within a specified width.

Example — assign the result to a variable:

```
text = "PYTHON"
result = text.center(20, "-")
print(result)
```

Real-time use: CLI headings and reports.

26. ljust()

Definition: Left-justifies a string to a specified width.

Example — assign the result to a variable:

```
name = "Vishwa"
result = name.ljust(15, ".")
print(result)
```

Real-time use: Simple text tables.

27. rjust()

Definition: Right-justifies a string to a specified width.

Example — assign the result to a variable:

```
amount = "5000"
result = amount.rjust(10)
print(result)
```

Real-time use: Terminal reports.

28. zfill()

Definition: Pads a string on the left with zeros to reach a width.

Example — assign the result to a variable:

```
employee_no = "45"
employee_id = employee_no.zfill(6)
print(employee_id) # 000045
```

Real-time use: Employee IDs/invoice numbers.

29. expandtabs()

Definition: Replaces tab characters with spaces.

Example — assign the result to a variable:

```
text = "Name\tRole"
result = text.expandtabs(12)
print(result)
```

Real-time use: Readable tab-separated output.

5. STRING METHODS — VALIDATION

30. encode()

Definition: Encodes a string into bytes using an encoding.

Example — assign the result to a variable:

```
text = "Python"
data = text.encode("utf-8")
print(data)
```

Real-time use: Network, files and byte APIs.

31. isascii()

Definition: Checks whether all characters are ASCII.

Example — assign the result to a variable:

```
value = "Python123"
result = value.isascii()
print(result) # True
```

Real-time use: ASCII-only input validation.

32. isdecimal()

Definition: Checks whether all characters are decimal characters.

Example — assign the result to a variable:

```
value = "12345"
result = value.isdecimal()
print(result) # True
```

Real-time use: Strict decimal-text validation.

33. isdigit()

Definition: Checks whether all characters are digit characters.

Example — assign the result to a variable:

```
value = "12345"
result = value.isdigit()
print(result) # True
```

Real-time use: Numeric input validation.

34. isnumeric()

Definition: Checks whether all characters are numeric characters.

Example — assign the result to a variable:

```
value = "12345"
result = value.isnumeric()
print(result) # True
```

Real-time use: Broader numeric-text checks.

35. isalpha()

Definition: Checks whether all characters are alphabetic.

Example — assign the result to a variable:

```
name = "Vishwanath"
result = name.isalpha()
print(result) # True
```

Real-time use: Alphabet-only validation.

36. isalnum()

Definition: Checks whether all characters are letters or numbers.

Example — assign the result to a variable:

```
user_id = "Vishwa123"  
result = user_id.isalnum()  
print(result) # True
```

Real-time use: Username/ID validation.

37. isspace()

Definition: Checks whether all characters are whitespace and the string is non-empty.

Example — assign the result to a variable:

```
value = "  
result = value.isspace()  
print(result) # True
```

Real-time use: Detecting whitespace-only input.

38. islower()

Definition: Checks whether cased characters are all lowercase.

Example — assign the result to a variable:

```
value = "python"  
result = value.islower()  
print(result) # True
```

Real-time use: Checking normalized text.

39. isupper()

Definition: Checks whether cased characters are all uppercase.

Example — assign the result to a variable:

```
value = "PYTHON"  
result = value.isupper()  
print(result) # True
```

Real-time use: Checking normalized text.

40. istitle()

Definition: Checks whether the string follows title-case rules.

Example — assign the result to a variable:

```
value = "Python Programming"  
result = value.istitle()  
print(result) # True
```

Real-time use: Validating headings/names.

6. REAL-TIME MINI PROJECT — STUDENT REGISTRATION

```
name = input("Enter your name: ")
email = input("Enter your email: ")
phone = input("Enter phone: ")

clean_name = name.strip().title()
clean_email = email.strip().lower()
clean_phone = phone.strip().replace("-", "")

print("\n--- STUDENT DETAILS ---")
print("Name :", clean_name)
print("Email :", clean_email)
print("Phone :", clean_phone)
```

Methods used: strip(), title(), lower(), replace(). This is a practical input/data-cleaning workflow.

7. REAL-TIME MINI PROJECT — SECURITY LOG ANALYSIS

```
log = "ERROR: User authentication failed"

parts = log.split(":", 1)
log_type = parts[0]
message = parts[1].strip()
is_error = log.startswith("ERROR")

print("Type :", log_type)
print("Message :", message)
print("Alert :", is_error)
```

Methods used: split(), strip(), startswith(). Excellent for DevOps, SRE and cybersecurity examples.

8. MUST-REMEMBER COMPARISONS

find() → returns -1 when absent. index() → raises ValueError when absent.
split() → String → List. join() → List/iterable → String.
Strings are immutable → assign the returned value when you want to keep the transformation.

9. PRACTICE LABS

① Clean a customer name. ② Normalize an email. ③ Remove phone hyphens. ④ Extract email username/domain. ⑤ Validate .csv/.pdf files. ⑥ Count ERROR entries. ⑦ Extract a filename from a path. ⑧ Join a list of skills. ⑨ Generate zero-padded employee IDs. ⑩ Build a mini log parser.

VishwaTech-Labs • Python Training Notes